CS6370 Assignment 1

Name: Pranav Rao Roll No: DA25M022

Part 1:

1. **Suggest a simplistic top-down approach to sentence segmentation for English texts.**

We can simply use the regex pattern : **[\.\?\!]+** to split the text into sentences. This splits the sentence on sentence-ending punctuations such as full-stop, question mark and exclamation mark.

Do you foresee issues with your proposed approach in specific situations? Provide supporting examples and possible strategies that can be adopted to handle these issues.

This method springs up some issues such as:

a. splits titles like Dr. , Mr. , Ph.D.
b. splits decimals like $15.65
c. splits url like iitm.ac.in
d. some slangs could have exaggerated phrases like What????

To tackle this, we need more sophisticated regex techniques, such as defining the full-stop as two separate punctuations - separator in $15.64 and iitm.ac.in and ender as we need.

NLTK is one of the most commonly used packages for Natural Language Processing. What does the Punkt Sentence Segmenter in NLTK do differently from a simple top-down (rule-based) sentence segmentation approach?

What Punkt Sentence Segmenter does:
It uses a language-independent unsupervised algorithm for Abbreviation Detection, which is treated as a collocation detection problem.

It first performs Abbreviation detection by scanning the raw text to find tight collocation between a word and a following period. Then it uses Log Likelihood Ratios to determine if a word and a period occur together more frequently than by chance. Some scaling factors are applied (heuristically determined, such as for short lengths, words with internal periods, and capital words).

It then decides on the Sentence Boundary Decision based on token classification, orthographic context and collocation heuristics.

Additionally, compare both of the above approaches with spaCy's sentence segmentation method.

What Spacy Sentence Segmenter does:
Sentence Segmentation is performed as a subprocess of the dependency parsing of the raw text. It is strictly defined by the connected components of the dependency graph.

First, the parser builds the dependency graph (which can also be regarded as a forest of multiple dependency trees) token by token. Each token must have exactly one head (or parent) except the root of the tree along with its dependency label. A sentence in this context is the collection / span of tokens that form within a root token (essentially, the parser separates the current sequence of tokens as a sentence as soon as it finds a root token). Thus, the sentence is defined not by lexical markers, but syntatic completion. The decision of token labels is done by a neural network.

2. **Design an adversarial test suite for sentence segmentation:**
    a. **Create 15 sentences that are likely to break a naive segmentation system.**
    b. **Categorise them by error type (e.g., abbreviations, decimals, ellipsis, quotes, multiple punctuation, etc.).**

Adversarial Test Suite:

a. URL:
    i. iitm.ac.in
b. Abbreviations:
    i. Ph.D.
    ii. This paper was written by Grover et al. and was published in 1992.
c. Decimals:
    i. You owe me $15.32 for those biscuits.
d. Ellipsis:
    i. Uh…Sure, we can try to go to Disneyland tomorrow.
e. Quotes:
    i. And He said, "What a wonderful world!"
    ii. "Stop!", he yelled. "You have got it all wrong."
f. Multiple Punctuation:
    i. What??????
    ii. What is wrong with you?!
    iii. I saw the movie (which, by the way, was terrible!) and went shopping.
    iv. What were her vacation plans anyway, huh? Tennis? Beach? Or the movies?
    v. The author seemed to have lived between 143? - 1450.
g. Emoticons:
    i. <|:?)>
h. Proper names:
    i. Would you like to play Jeopardy! with me?
    ii. .hack//SIGN is such a great show!

Evaluate how many errors each segmenter makes on your test suite.

Naive Sentence Segmentor: 0.0 %
Punkt Sentence Segmentor: 26.67%
Spacy Sentence Segmentor: 86.67%

Part 2

Adversial Test Suite for Tokenization:

1. ?)
    a. Expected: [?,)]
2. nlp_student@iitm.ac.in
    a. Expected: [nlp_student@iitm.ac.in]
3. I'm
    a. Expected: [I'm]
4. 3.14
    a. Expected: [3.14]
5. Good muffins cost $3.88 (roughly 3,36 euros)\n in New York. Please buy me \n two of them.\nThanks.
    a. Expected: [Good, muffins, cost, $, 3.88, (, roughly, 3,36, euros,), \n, in, New, York, . , Please, buy, me, \n, two, of, them, . , \n, Thanks, .]
6. The meeting is at 14:30 12-03-2025
    a. Expected: [The, meeting, is, at, 14:30, 12-03-2025]
7. 1999/2000
    a. Expected: [1999 , / , 2000]
8. The model achieved 95%accuracy.Furthermore, it…
    a. Expected: [The, model, achieved, 95, %, accuracy, ., Furthermore, it, …]

1. **Suggest a simplistic top-down approach for tokenization in English text.**

We can use a simple regex expression as **\w+|[^\w\s]+** which matches any word or a non-word character

Identify specific situations where your proposed approach may fail to produce expected results.
This tokenizer fails in 1 - 6 cases of the suite.

2. **Study about NLTK's Penn Treebank tokenizer and spaCy's tokenizer. What type of knowledge does each of them use — Top-down, Bottom-up, or a hybrid approach? Clearly justify your answer for both tokenizers.**

Penn Treebank Tokenizer -

It is a rule-based tokenizer that uses regex patterns to obtain tokens. It is based on the set of conventions used for the Penn Treebank corpus. It opens contradictions, handles punctuation, end-of-line periods, etc. It assumes that the text has already been split into sentences.

Spacy Tokenizer -
Spacy applies certain language-specific rules to the text to split it into tokens. It is a top-down, rule-based tokenizer. First, the text is split according to whitespace. Then it is processed from left to right. On each substring obtained, it performs two checks: is the substring an exception (such as *don't* and *U.K.*) and whether a prefix, suffix and an infix can be split off. The tokenizer then applies the same loop (recursively) to the resulting substrings so that it can split complex, nested tokens like a combination of abbreviations and multiple punctuation marks.

3. **Perform word tokenization of the sentence-segmented documents using:**
    a. **The proposed top-down method**
    b. **Penn Treebank Tokenizer**
    c. **spaCy Tokenizer**

    State a possible scenario along with an example where:
    a. **Your approach performs better than the Penn Treebank and spaCy tokenizers**
    b. **Your approach performs worse than the Penn Treebank and spaCy tokenizers**

From the adversarial test suite,
Penn Tokenizer fails at: 6 - 7
Spacy Tokenizer fails at: 7 - 8

Part 3

1. **What is the difference between stemming and lemmatization? Clearly explain the conceptual differences between the two approaches in terms of linguistic knowledge, output form, and computational complexity. Give suitable examples to illustrate your answer.**

Stemming	**Lemmatization**
Heuristically removes all derivational affixes	Returns base form of a word using a dictionary
Resulting word may not be a meaningful word	Resulting word will be a meaningful word
Context Independent	Context Aware

Quick and less complex	Slow and more complex
Example : 1. saw -> s 2. studies -> studi 3. communication -> communi	Example : 1. saw -> see 2. studies -> study 3. communication -> communicate
Increases recall of IR systems while decreases precision	Decreases recall of IR systems while increases precision

4. **Using the word-tokenised text from the Cranfield Dataset:**
    a. **Perform stemming using Porter's Stemmer**
    b. **Perform lemmatization using WordNet Lemmatizer**

    Compare the outputs of the two methods by analyzing:
    a. **Changes in vocabulary size**
    b. **Examples of over-stemming (if any)**
    c. **Cases where lemmatization preserves semantic meaning better**

Original Vocabulary size: 9765
Vocabulary size after stemming : 7125
Vocabulary size after lemmatizing : 8386

So vocabulary shrinks for both stemming and lemmatizing, which is expected as they reduce all inflections on the words. It shrinks more for stemming as it removes all inflections of a word to the word stem (which may not be appropriate for the context).

Example of an over-stemming -
Original - ['an', 'empirical', 'evaluation', 'of', 'the', 'destalling', 'effects', 'was', 'made', 'for']
Stemming - ['an', 'empir', 'evalu', 'of', 'the', 'destal', 'effect', 'wa', 'made', 'for']
Lemmatizing - ['an', 'empirical', 'evaluation', 'of', 'the', 'destalling', 'effect', 'be', 'make', 'for']

Here, the semantic meaning is also preserved better for lemmatizing

Part 4

1. **Remove stopwords from the tokenized documents using a curated list, such as the stopword list provided by NLTK.**

2. **Suggest a bottom-up (data-driven) approach for constructing a stopword list that is specific to the given corpus of documents. Clearly explain the intuition behind your proposed method.**

a. Collect all tokens in the documents

b. Prepare a frequency table of all tokens within all docs (after appropriate pre-processing such as trimming of white spaces and lower case)
c. Subtract the frequencies with the value of the highest frequency
d. Convert the relative frequencies into probabilities using softmax method (by subtracting the highest frequency value we are essentially performing the Log-Sum-Exp trick)
e. Set a threshold value
f. Remove all words which appear with a probability above the threshold

The logic behind this was stopwords are words which appear more frequently and provide little information to the context. Thus, we look into the probabilities of their occurrences (not frequency as frequency of occurrence is more context length depended) and truncate the words which have high probability

3. **Implement the strategy proposed in the previous question and compare the stopwords obtained with those provided by NLTK on the Cranfield dataset. Your comparison should include:**
    a. **Number of stopwords identified**
    b. **Overlap between the two lists**
    c. **Words that appear in one list but not the other**

Original Reduced text length : 239136
Stop words removed from NLTK: 92959
Stop words removed from Data driven method : 114237
Overlap between the two stopwords: 87
Unique words in NLTK stopwords : 111
Unique words in Data stopwords : 1948

Part 5

1. **Given a set of queries Q and a corpus of documents D, what would be the total number of similarity computations required to estimate the similarity between every query and every document? Assume that TF-IDF vector representations are already available for both queries and documents over a vocabulary V . Clearly express your answer in terms of |Q|, |D|, and |V |.**

Total number of similarity computations required without TF-IDF: $\mathcal{O}(|Q| \times |D|)$

Required Query - Document matrix \in R(|Q| x |D|):
Represent queries in the word space:
A = Term-Query matrix \in R(|Q| x |V|)
Represent documents in the word space:
B = Term-Document matrix \in R(|D| x |V|) matrix

C = AB^T

Total number of similarity computations required with TF-IDF: $\mathcal{O}(|Q| \times |D| \times |V|)$

2. **Suggest how the concept of an inverted index can be used to reduce the time complexity derived in the previous question. You may introduce additional variables (such as average document frequency or sparsity factors) to explain your reasoning. Provide a clear comparison between the naive approach and the inverted-index-based approach in terms of computational complexity.**

We can have inverted index of terms from the vocabulary of list of documents which contain that term and its term weight.

If L_q is the average length of a query, $df(t)$ is the number of documents containing a term, df_{avg} is the average length of a posting length then,

1. For each token in the query, fetch the posting list
2. Find the union of the posting list
3. Perform the similarity computation with the Query and the retrieved document

Since there are only a few documents that actually are relevant to a query, this reduces the computation time.
Time complexity:
$\mathcal{O}(|Q| \times L_q \times df_{avg})$